

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Направление подготовки: 09.03.04 – Системное и прикладное программное обеспечение

Дисциплина «Алгоритмы и структуры данных»

Отчёт по блоку задач №2

Выполнил
Линейский Аким Евгеньевич
Р3215
Преподаватель
Тараканов Денис Сергеевич

Санкт-Петербург, 2026

1. Задача Е. Коровы в стойла

1.1. Условие задачи

Е. Коровы в стойла

✓ Полное решение

Ограничение времени	0.1 секунда
Ограничение памяти	256 Мб
Ввод	стандартный ввод или input.txt
Вывод	стандартный вывод или output.txt

На прямой расположены стойла, в которые необходимо расставить коров так, чтобы минимальное расстояние между коровами было как можно больше.

Формат ввода

В первой строке вводятся числа N ($2 < N \leq 10^5$) – количество стойл и K ($1 < K < N$) – количество коров. Во второй строке задаются N натуральных чисел в порядке возрастания – координаты стойл (координаты не превосходят 10^9).

Формат вывода

Выведите одно число – наибольшее возможное допустимое расстояние.

Пример 1

Ввод

```
6 3
2 5 7 11 15 20
```

Вывод

```
9
```

Пример 2

Ввод

```
5 3
1 2 3 100 1000
```

Вывод

```
99
```

1.2. Идея решения

Для поиска максимального минимального расстояния между коровами применяется бинарный поиск по ответу. Область поиска задается на отрезке от нуля до разности координат между последним и первым стойлом.

На каждой итерации вычисляется среднее расстояние. Корректность текущего значения проверяется жадным алгоритмом в функции `is_placed`. Первая корова всегда отправляется в самое левое стойло, а каждая следующая размещается в ближайшем доступном стойле, которое удалено от предыдущего как минимум на искомую величину. Если удалось успешно расставить необходимое количество коров, текущий результат сохраняется и поиск продолжается в правой половине для увеличения дистанции. В противном случае область поиска сужается влево.

1.3. Исходный код программы

```
1 #include <iostream>
2 #include <vector>
3
4 bool is_placed(const std::vector<int> &coords, int k, int min_dist) {
5     int count = 1;
6     int last_pos = coords[0];
7
8     for (size_t i = 1; i < coords.size(); ++i) {
9         if (coords[i] - last_pos >= min_dist) {
10             count++;
11             last_pos = coords[i];
12         }
13     }
14     return count >= k;
15 }
16
17 int main() {
18     int n, k;
19     if (!(std::cin >> n >> k)) {
20         return 0;
21     }
22
23     std::vector<int> coords(n);
24     for (int i = 0; i < n; ++i) {
25         std::cin >> coords[i];
26     }
27
28     int left = 0;
29     int right = coords[n - 1] - coords[0];
30     int result = 0;
31
32     while (left <= right) {
33         int mid = left + (right - left) / 2;
34
35         if (is_placed(coords, k, mid)) {
36             result = mid;
37             left = mid + 1;
38         } else {
39             right = mid - 1;
40         }
41     }
42
43     std::cout << result << std::endl;
44
45     return 0;
46 }
```

2. Задача F. Число

2.1. Условие задачи

F. Число

✓ Полное решение

Ограничение времени	1 секунда
Ограничение памяти	256 Мб
Ввод	стандартный ввод или number.in
Вывод	стандартный вывод или number.out

Вася написал на длинной полоске бумаги большое число и решил похвастаться своему старшему брату Пете этим достижением. Но только он вышел из комнаты, чтобы позвать брата, как его сестра Катя вбежала в комнату и разрежала полоску бумаги на несколько частей. В результате на каждой части оказалось одна или несколько идущих подряд цифр.

Теперь Вася не может вспомнить, какое именно число он написал. Только помнит, что оно было очень большое. Чтобы утешить младшего брата, Петя решил выяснить, какое максимальное число могло быть написано на полоске бумаги перед разрезанием. Помогите ему!

Формат ввода

Входной файл содержит одну или более строк, каждая из которых содержит последовательность цифр. Количество строк во входном файле не превышает 100, каждая строка содержит от 1 до 100 цифр. Гарантируется, что хотя бы в одной строке первая цифра отлична от нуля.

Формат вывода

Выведите в выходной файл одну строку — максимальное число, которое могло быть написано на полоске перед разрезанием.

Пример 1

Ввод 	Вывод 
2 20 004 66	66220004

Пример 2

Ввод 	Вывод 
3	3

2.2. Идея решения

Задача составления максимального числа из отдельных строковых фрагментов решается с помощью жадной сортировки со специфическим компаратором. Обычная лексикографическая сортировка не подходит, так как строка 66 должна идти перед строкой 662, ведь число 66662 больше числа 66266.

Для правильного упорядочивания компаратор сравнивает суммы двух строк в разном

порядке соединения. Если комбинация первой строки со второй дает больший результат, чем комбинация второй строки с первой, то первая строка признается приоритетной. После упорядочивания всего массива векторов строки последовательно выводятся на экран, формируя максимально возможное итоговое число.

2.3. Исходный код программы

```
1 #include <algorithm>
2 #include <iostream>
3 #include <string>
4 #include <vector>
5
6 bool compareNumbers(const std::string &a, const std::string &b) {
7     return a + b > b + a;
8 }
9
10 int main() {
11     std::vector<std::string> parts;
12     std::string temp;
13
14     while (std::cin >> temp) {
15         parts.push_back(temp);
16     }
17
18     if (parts.empty()) {
19         return 0;
20     }
21
22     std::sort(parts.begin(), parts.end(), compareNumbers);
23
24     for (const std::string &s : parts) {
25         std::cout << s;
26     }
27     std::cout << std::endl;
28
29     return 0;
30 }
```

3. Задача G. Кошмар в замке

3.1. Условие задачи

G. Кошмар в замке

✓ Полное решение

Ограничение времени	2 секунды
Ограничение памяти	256 Мб
Ввод	стандартный ввод или aurora.in
Вывод	стандартный вывод или aurora.out

Ходят легенды, что пока Аврора спала, ей снилось, что она ходит по разным местам: леса, поля, города и сёла. И вот однажды она наткнулась на пещеру, в которой сидел мудрец. Когда мудрец поднял на Аврору глаза, он изрёк: «Дорогая Аврора! Ты уже годами скитаешься по этим землям. Я хочу предложить тебе задачку. Вот тебе строка s . Каждая буква из алфавита имеет свой вес c_i . Вес строки, которую ты можешь получить из s многократным обменом любых двух букв, вычисляется так: для каждой буквы алфавита посчитай максимальное расстояние между позициями, в которых стоит эта буква и перемножь его с весом этой буквы. Принеси мне строку максимально возможного веса, и я тебе расскажу, в чём смысл жизни».

К счастью, когда Аврора уже шла со строкой к мудрецу, её поцеловал Филипп, и Аврора вышла из этого кошмара. Теперь вам предлагается самим окунуться в этот кошмар и решить поставленную задачу.

Формат ввода

Дана строка, состоящая из строчных букв латинского алфавита ($1 \leq |s| \leq 10^5$). Следующая строка ввода содержит 26 чисел — веса букв латинского алфавита от «a» до «z», веса неотрицательны и не превосходят $2^{31} - 1$.

Формат вывода

Выведите строку s , в которой переставлены буквы так, чтобы полученный вес был максимально возможным. Если искомым вариантов несколько, выведите любой из них.

Пример

Ввод	Вывод
lkshrules 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16	slkhruels

3.2. Идея решения

Вес строки зависит от расстояния между крайними одинаковыми символами, поэтому выгодно разносить парные буквы с наибольшим весом как можно дальше к краям.

Сначала подсчитывается частота появления каждого символа во входной строке. Затем данные сортируются по убыванию приоритета, где главным критерием выступает вес буквы, а второстепенным является лексикографический порядок для детерминированности. Формирование строки происходит за один проход по упорядоченным символам. Если у символа есть пара, то по одной букве добавляется в начало и в конец результирующего палиндрома, а счетчик уменьшается на два. Все оставшиеся непарные символы собираются в центральной части строки. Из-за изначальной сортировки тяжелые буквы занимают

самые удаленные и выгодные позиции.

3.3. Исходный код программы

```
1 #include <algorithm>
2 #include <iostream>
3 #include <string>
4 #include <vector>
5
6 struct CharInfo {
7     char c;
8     long long weight;
9     int count;
10 };
11
12 bool compareWeights(const CharInfo &a, const CharInfo &b) {
13     if (a.weight != b.weight) {
14         return a.weight > b.weight;
15     }
16     return a.c < b.c;
17 }
18
19 int main() {
20     std::string s;
21     if (!(std::cin >> s)) {
22         return 0;
23     }
24
25     std::vector<long long> weights(26);
26     for (int i = 0; i < 26; ++i) {
27         std::cin >> weights[i];
28     }
29
30     std::vector<int> counts(26, 0);
31     for (char c : s) {
32         counts[c - 'a']++;
33     }
34
35     std::vector<CharInfo> info;
36     for (int i = 0; i < 26; ++i) {
37         if (counts[i] > 0) {
38             info.push_back({static_cast<char>('a' + i), weights[i], counts[i]});
39         }
40     }
41
42     std::sort(info.begin(), info.end(), compareWeights);
43
44     std::string left_side = "";
45     std::string right_side = "";
46     std::string middle = "";
47
48     for (auto &item : info) {
49         if (item.count >= 2) {
50             left_side += item.c;
51             right_side = item.c + right_side;
52             item.count -= 2;
53         }
54         while (item.count > 0) {
55             middle += item.c;
56             item.count--;
```

```
57     }  
58 }  
59  
60 std::cout << left_side << middle << right_side << std::endl;  
61  
62 return 0;  
63 }
```


4. Задача Н. Магазин

4.1. Условие задачи

Н. Магазин

✓ Полное решение

Ограничение времени	2 секунды
Ограничение памяти	256 Мб
Ввод	стандартный ввод или shop.in
Вывод	стандартный вывод или shop.out

У Билла большая семья: трое сыновей, девять внуков. И всех надо кормить. Поэтому Билл раз в неделю ходит в магазин.

Однажды Билл пришел в магазин и увидел, что в магазине проводится акция под названием «каждый k -й товар бесплатно». Изучив правила акции, Билл выяснил следующее. Пробив на кассе товары, покупатель получает чек. Пусть в чеке n товаров, тогда n/k округленное вниз самых дешевых из них достаются бесплатно.

Например, если в чеке пять товаров за 200, 100, 1000, 400 и 100 рублей, соответственно, и $k = 2$, то бесплатно достаются оба товара по 100 рублей, всего покупатель должен заплатить 1600 рублей.

Билл уже выбрал товары, и направился к кассе, когда сообразил, что товары, которые он хочет купить, можно разбить на несколько чеков, и благодаря этому потратить меньше денег.

Помогите Биллу выяснить, какую минимальную сумму он сможет заплатить за выбранные товары, возможно разбив их на несколько чеков.

Формат ввода

Первая строка входного файла содержит два целых числа n, k ($1 \leq n \leq 100\,000, 2 \leq k \leq 100$) — количество товаров, которые хочет купить Билл и параметр акции «каждый k -й товар бесплатно».

Следующая строка содержит n целых чисел a_i ($1 \leq a_i \leq 10\,000$) — цены товаров, которые покупает Билл.

Пример

Ввод 	Вывод 
5 2 200 100 1000 400 100	1300

4.2. Идея решения

Для минимизации стоимости покупок необходимо сделать бесплатными как можно более дорогие товары. Условие акции позволяет получить бесплатно каждый k -й товар в чеке, при этом бесплатными становятся самые дешевые позиции.

Жадный алгоритм заключается в сортировке всех цен в порядке убывания. Разбиение на оптимальные чеки моделируется простым проходом по упорядоченному массиву. Каждый k -й по счету элемент пропускается, так как его можно сделать бесплатным, объединив в один чек с предыдущими $k - 1$ более дорогими товарами. Цены всех остальных товаров суммируются в итоговую стоимость.

4.3. Исходный код программы

```
1 #include <algorithm>
2 #include <iostream>
3 #include <vector>
4
5 int main() {
6     int n, k;
7     if (!(std::cin >> n >> k)) {
8         return 0;
9     }
10
11     std::vector<int> prices(n);
12     for (int i = 0; i < n; ++i) {
13         std::cin >> prices[i];
14     }
15
16     std::sort(prices.begin(), prices.end(), std::greater<int>());
17
18     long long total = 0;
19     for (int i = 0; i < n; ++i) {
20         if ((i + 1) % k == 0) {
21             continue;
22         }
23         total += prices[i];
24     }
25
26     std::cout << total << std::endl;
27
28     return 0;
29 }
```